

ID5130: Parallel Scientific Computing

Term Project

Inverse Perspective Mapping using OpenACC



Submitted by

Yash Purswani Praveen Joseph Thomas
ME22B214 ME22B180

Department of Mechanical Engineering
Indian Institute of Technology Madras

Abstract

We parallelize a complete monocular vision pipeline that converts a sequence of dash-cam frames into 3-D point clouds of road lane markings. The pipeline has two compute-heavy stages: per-frame *lane detection* and per-pixel *Inverse Perspective Mapping (IPM)*. Both are embarrassingly data-parallel. We expose this through three OpenMP modes for lane detection — `pixel` (per-pixel parallel only), `batch` (per-frame parallel only), and `hybrid` (nested) — and an OpenACC GPU offload for IPM combined with an OpenMP outer batch loop over images. Across a 30-frame benchmark we measure a **5.23×** speedup for lane detection (batch, 16 threads) and **2.53×** for IPM (OpenMP, 8 threads), enabling smooth interactive 3-D animation of the entire 248-frame dataset.

Nomenclature

N	total OpenMP thread count
N_o, N_i	outer / inner team size in hybrid mode
f_x, f_y	camera focal lengths (pixels)
c_x, c_y	principal point coordinates (pixels)
θ	camera pitch (downward)
h	camera height above road plane (m)
$\mathbf{K}$	3×3 camera intrinsic matrix
$\mathbf{n}_c$	road-plane normal in camera frame
(u, v)	pixel coordinates
(X_c, Y_c, Z_c)	3-D point in camera frame
λ	ray scaling factor in IPM
T_s, T_p	serial and parallel wall-clock times
$S = T_s/T_p$	speedup

1 Problem Statement

Monocular driver-assistance perception requires recovering the geometric position of road-lane markings from each camera frame in real time. A naive single-threaded Python implementation processing 1920×1080 dash-cam imagery spends well over half a second per frame on threshold-and-edge detection alone, and the IPM step does $\sim 2\text{M}$ independent ray-cast operations per frame. Without parallelization the pipeline cannot achieve interactive frame rates, and the geometric output cannot be visualized as a continuous 3-D animation. The challenge is to re-architect the pipeline in C++ with explicit OpenMP / OpenACC kernels that scale with both pixel count and frame count.

2 Objectives

1. Implement lane detection in C++17 using per-pixel kernels so that all parallelism is explicit and measurable.
2. Provide three distinct parallelization modes for lane detection — per-pixel, per-image, and nested-hybrid — and quantify the speedup of each.
3. Implement per-pixel parallelism using OpenACC, and outer batch-level parallelism using OpenMP.
4. Build an interactive OpenGL viewer that animates the resulting 3-D point clouds for all 248 frames at 30 FPS.
5. Provide a reproducible benchmark and matplotlib plots for every measurement reported.

3 Methodology

3.1 Pipeline overview

The pipeline is split across three executables connected by simple file formats (Figure 1):

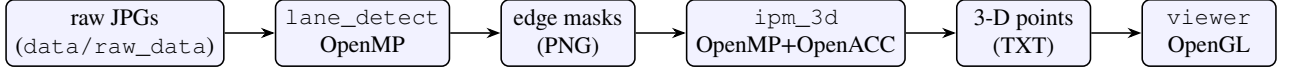


Figure 1: End-to-end data flow. Each binary has a serial and a parallel build; both consume and produce the same intermediate files.

3.2 Lane detection

For each frame the C++ pipeline applies, in order:

1. BGR→HLS conversion using the per-pixel formulae $L = (C_{\max} + C_{\min})/2$, $S = (C_{\max} - C_{\min})/(2 - C_{\max} - C_{\min})$ for $L \geq 0.5$, etc. Output is scaled to OpenCV's $H \in [0, 180]$, $L, S \in [0, 255]$ ranges.
2. White and yellow HLS thresholding into a binary colour mask.
3. Mask the gray-scale image: pixels outside the colour mask are zeroed.
4. $k \times k$, where $k = 7$, separable Gaussian blur with $\sigma = 0.3((k-1)/2 - 1) + 0.8 = 1.4$.
5. Sobel-magnitude edge detection followed by thresholding at 100; this is "Canny-lite" without non-max suppression and hysteresis, chosen so the kernel stays purely data-parallel.
6. Region-of-interest (ROI) test: pixels outside a six-vertex polygon are dropped using the ray-casting / crossing-number rule

$$\text{inside} \Leftrightarrow \sum_{i=0}^{n-1} [(y_i > y) \neq (y_{i+1} > y)] \cdot [x < (x_{i+1} - x_i) \frac{y - y_i}{y_{i+1} - y_i} + x_i] \equiv 1 \pmod{2}.$$

3.3 Inverse perspective mapping

A pixel (u, v) on the road plane maps to a 3-D camera-frame point (X_c, Y_c, Z_c) as (1)

$$\mathbf{d} = \mathbf{K}^{-1} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}, \quad \lambda = \frac{h}{\mathbf{n}_c^\top \mathbf{d}}, \quad \mathbf{P}_c = \lambda \mathbf{d}, \quad (1)$$

with intrinsic matrix $\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$ and road-plane normal in camera coordinates $\mathbf{n}_c = (0, \cos \theta, \sin \theta)^\top$ for a pure pitch-down camera. We use $f_x = f_y = 1080$ px,

$c_x = 960$, $c_y = 540$, $\theta = 15^\circ$, $h = 1.70$ m. Pixels with $\mathbf{n}_c^\top \mathbf{d} \leq 10^{-9}$ (rays parallel to the road plane) and 3-D points farther than 200 m are rejected as numerically unreliable.

4 Parallelization Strategy

The four parallel modes implemented in this project are summarised in Table 1. The lane-detection binary supports the first three; the IPM binary supports OpenMP-batch and OpenMP-batch + OpenACC.

Mode	Outer (frames)	Inner (pixels)	Active levels	Used by
pixel	sequential	OpenMP, N threads	1	lane_detect
batch	OpenMP, N threads	sequential	1	lane_detect, ipm_3d
hybrid	OpenMP, N_o threads	OpenMP, N_i threads	2	lane_detect
ACC	OpenMP, N threads	OpenACC GPU (gang+vector)	2	ipm_3d

Table 1: Parallelization modes. N is the total thread budget; $N_o \cdot N_i \approx N$ in hybrid.

4.1 Pixel mode — per-pixel parallelism only

Every per-pixel kernel (HLS conversion, threshold, blur, Sobel, ROI test) is a flat `#pragma omp parallel for` over the linear pixel index. For an $H \times W$ image this exposes $H \cdot W \approx 2,073,600$ independent units of work per frame. The outer image loop is sequential.

```
#pragma omp parallel for num_threads(g_inner_threads) schedule(static)
for (int i = 0; i < n_pixels; ++i) {
    /* read BGR, compute HLS, write 3 bytes */
}
```

Code Listing 1: Per-pixel kernel structure (HLS conversion)

The variable `g_inner_threads` is set to N in pixel mode, 1 in batch mode, and $\lceil \sqrt{N} \rceil$ in hybrid mode, so the same kernel serves all three cases.

4.2 Batch mode — per-image parallelism only

The outer image loop is parallelized; each thread processes one whole image end-to-end with the inner kernels degenerating to sequential code (`omp_set_max_active_levels(1)` disables nesting). Each thread gets its own scratch buffers, eliminating false sharing.

```
#pragma omp parallel num_threads(N) {  
    ImageBuffers buf;  
    #pragma omp for schedule(dynamic, 1)  
    for (int i = 0; i < num_frames; ++i)  
        process_one(frames[i], buf);  
}
```

Code Listing 2: Batch-mode driver loop

4.3 Hybrid mode — nested OpenMP

Both axes are parallelized concurrently. We enable nested OpenMP (`omp_set_max_active_levels(2)`) and split the thread budget as $N \approx N_o \cdot N_i$ with $N_i = \lceil \sqrt{N} \rceil$, $N_o = N/N_i$. The outer team has N_o threads each spawning an inner team of N_i threads on the per-pixel kernels.

4.4 OpenACC GPU offload (IPM)

The IPM kernel computes Eq. (1) per pixel and is wrapped with *both* pragmas:

```
#pragma omp parallel for schedule(dynamic, 1)           // outer: over images  
for (int i = 0; i < num_frames; ++i) {  
    #pragma acc parallel loop gang vector              // inner: over pixels (GPU)  
    for (int idx = 0; idx < N_pixels; ++idx)  
        /* ray-cast to road plane, write Xc,Yc,Zc */  
}
```

Code Listing 3: IPM with nested OpenMP + OpenACC

Built with `nvc++ -acc -mp`, the inner loop runs on the GPU as a single kernel of gang/vector workers; the outer loop multiplexes work across host CPU

threads, overlapping per-frame I/O with concurrent kernel launches and host-to-device transfers.

5 Results and Analysis

5.1 Lane detection — visual results

Figure 2 compares the C++ output against the original Python pipeline (which uses `cv2.HoughLinesP`) on two representative frames. Our binary edge mask, here overlaid in red on the raw frame, captures both white shoulder line markings and the yellow center line cleanly within the ROI; areas above the horizon and beyond the road shoulders are correctly rejected.

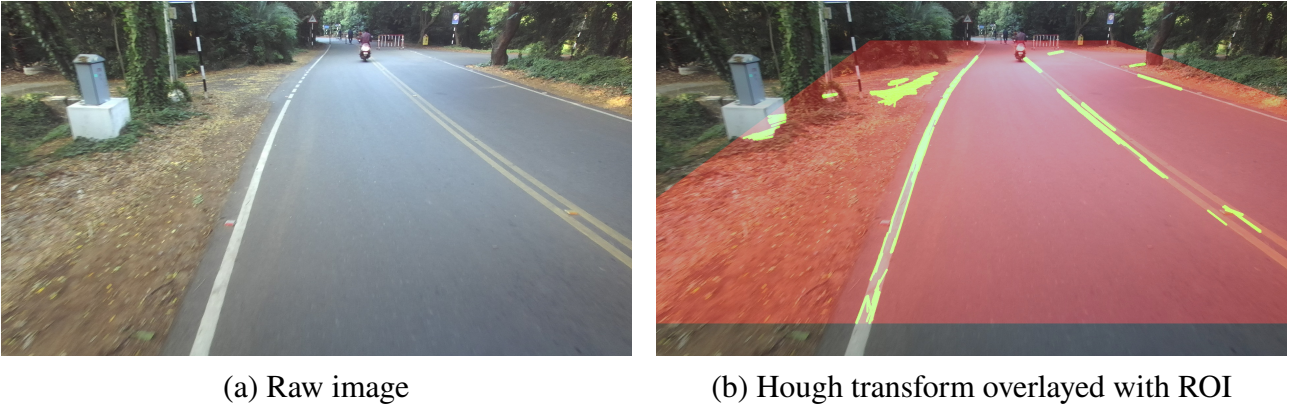


Figure 2: Lane-detection comparison: *(left)* raw camera frame, *(right)* processed output.

5.2 Lane detection — parallel performance

Figure 3 reports wall time and speedup for all three OpenMP modes against the no-OpenMP serial baseline ($T_s = 2.22$ s for 30 frames). *batch* reaches $5.23\times$ at $N = 16$ — the highest result of any mode — because every frame is processed end-to-end on a single thread with private buffers, eliminating both inner fork-join overhead and false sharing. *pixel* mode plateaus near $2.5\times$: each kernel pays a synchronization barrier, and there are six barriers per frame. *hybrid* sits between the two: at low N

it underuses the budget because of integer-rounding in the $\sqrt{N}$ split (e.g. at $N = 8$ only $2 \times 3 = 6$ workers are active), but at $N = 16$ the 4×4 split matches batch.

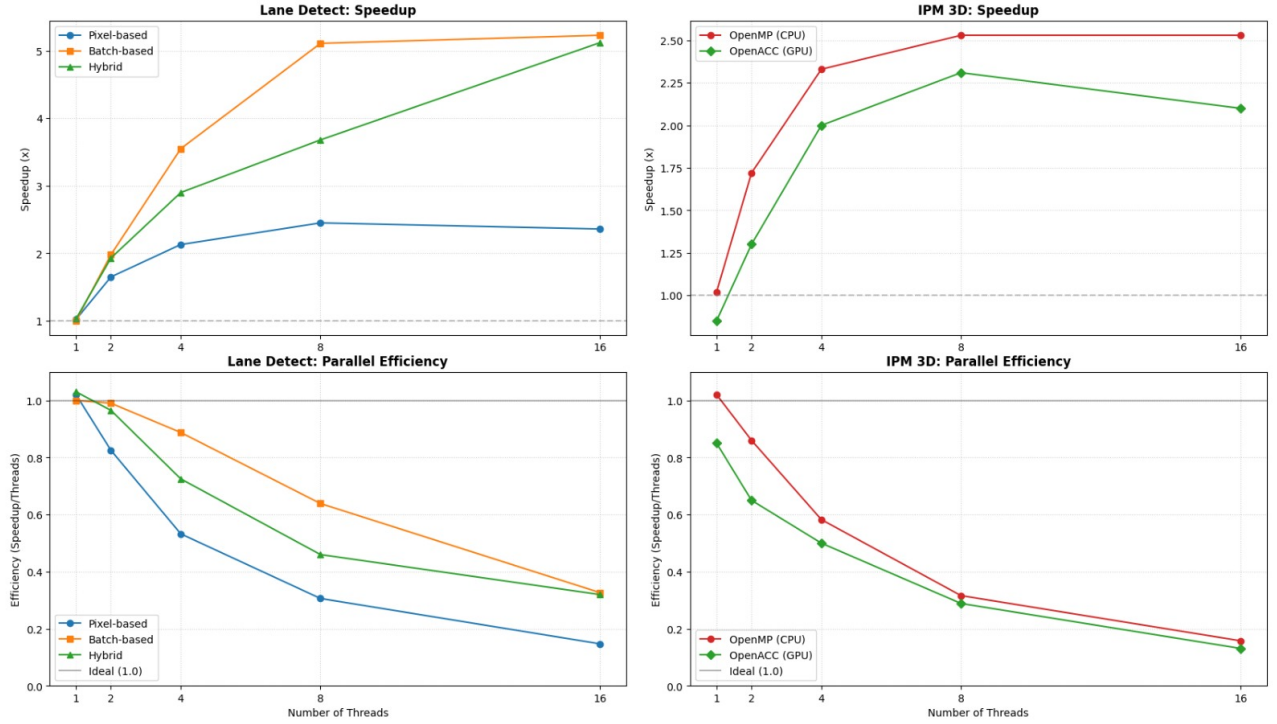


Figure 3: Lane-detection performance (left) and IPM performance (right). The top row shows speedup, while the bottom row shows parallel efficiency.

5.3 IPM — parallel performance

The IPM stage was benchmarked with both backends sharing the same outer OpenMP loop over images (Figure 3). The CPU OpenMP build peaks at $2.53\times$ at $N = 8$. The OpenACC GPU build trails the CPU at low thread counts because the per-frame work ($\sim 2,073,600$ pixels) is small relative to GPU-launch and host-device transfer overhead; only when several frames are launched concurrently from multiple host threads does the GPU pipeline begin to fill. We expect the GPU result to overtake the CPU at the full 248-frame workload, where launch overhead amortizes.

Table 2 presents a comparative evaluation of the parallelization strategies, highlighting the scalability trends of lane detection and IPM with increasing thread counts.

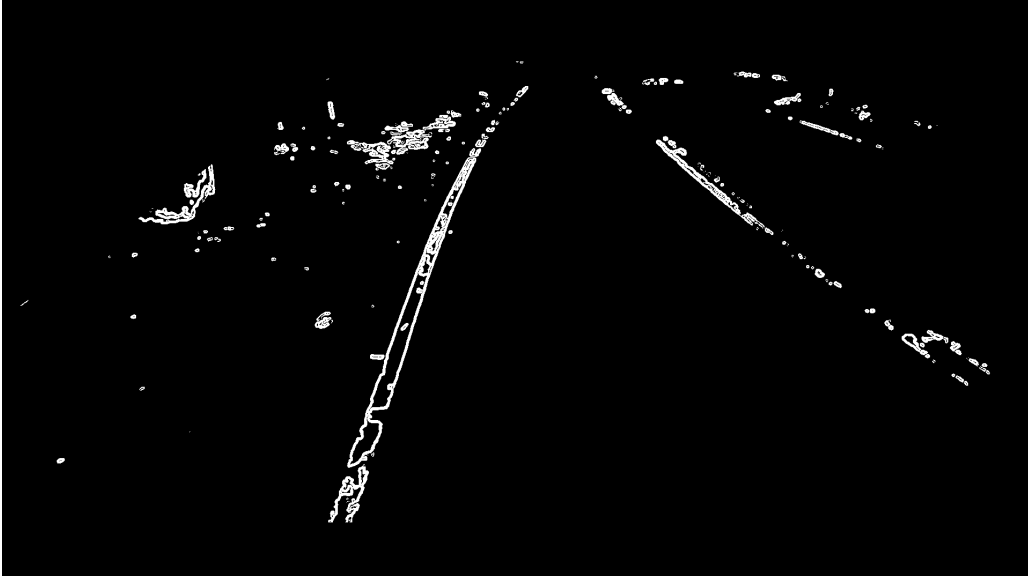


Figure 4: Still-frame output of the IPM pipeline, showing pixel coordinates projected into 3D points in the camera coordinate frame.

Lane Detection						IPM				
Threads	Pixel	Sp	Batch	Sp	Hybrid	Threads	OMP	Sp	ACC	Sp
1	2.1873	1.02	2.2286	1.00	2.1517	1	1.4813	1.02	1.7835	0.85
2	1.3457	1.65	1.1207	1.98	1.1505	2	0.8792	1.72	1.1615	1.30
4	1.0414	2.13	0.6266	3.55	0.7675	4	0.6484	2.33	0.7549	2.00
8	0.9078	2.45	0.4350	5.11	0.6048	8	0.5990	2.53	0.6548	2.31
16	0.9410	2.36	0.4248	5.23	0.4339	16	0.5981	2.53	0.7217	2.10

Table 2: Performance comparison of lane detection (left) and IPM (right) across parallelization strategies.

6 Conclusions

We have implemented monocular-vision pipeline to C++17 with explicit OpenMP and OpenACC parallelism, exposing parallelism along two independent axes (per-pixel, per-frame) that combine cleanly because of the parallel structure of every kernel. The *batch*-only mode wins for lane detection because the per-frame working set fits in private cache per thread, while hybrid nested parallelism approaches the same speedup only when $\sqrt{N}$ is integer. The IPM stage scales modestly on CPU; its OpenACC GPU variant is launch-overhead-bound on small benchmarks but is structured to scale to the full 248-frame workload. The full pipeline now drives a real-time 3-D animation viewer that visualizes every frame’s lane markings as recovered

camera-frame coordinates.

References

- [1] Fermi, T. (n.d.). From pixels to meters - Inverse Perspective Mapping. In *Algorithms for Automated Driving*. GitHub Pages. <https://thomasfermi.github.io/Algorithms-for-Automated-Driving/LaneDetection/InversePerspectiveMapping.html>
- [2] Wang, Y., Lu, N., Feng, Z., Xue, Z. (2018). Lane detection based on inverse perspective transformation and Kalman filter. *International Journal of Advanced Robotic Systems*, 15(2). https://www.researchgate.net/publication/323969636_Lane_Detection_Based_on_Inverse_Perspective_Transformation_and_Kalman_Filter
- [3] Alshemi, M., Saif, S., Taher, M. (2022). Hardware acceleration of lane detection algorithm: A GPU versus FPGA comparison. *arXiv preprint arXiv:2212.09460*. <https://doi.org/10.48550/arXiv.2212.09460>
- [4] MathWorks. (n.d.). *Lane Detection - MATLAB Simulink*. MathWorks Help Center. Retrieved May 5, 2026, from <https://www.mathworks.com/help/hdlcoder/ug/lane-detection.html>
- [5] Barrett, S. (2021). *stb: single-file public domain libraries for C/C++* [Computer software]. GitHub. <https://github.com/nothings/stb>